

Extended Abstract

Motivation Training agents effectively in multi-agent adversarial games can be quite challenging because rewards are sparse, it’s hard to find same-level opponent, naive self-play often stalls early, and mixed interactions and complex group dynamics are hard to explore. In this work, we would like to address these challenges and explore an efficient training recipe, which could be applicable to many other problems and real-life scenarios.

Method We first implemented three RL algorithms from scratch and investigated their performances under different strategies including REINFORCE, discrete SAC and Proximal Policy Optimization (PPO). Also, we introduced a staged training curriculum:

Stage 1 Focusing on a single striker while keeping the other players inactive. This step allows the agent to gain a less nosier reward and solve a simpler task to begin with.

Stage 2 Training both the striker and the goalie of one team simultaneously with random opponents. We utilized combined team rewards to allow specialized learning of strikers and goalies.

Stage 3 Self-play. We intentionally brought various randomness (ϵ -Greedy) to the other team agent’s action to enhance exploration and prevent overfitting to specific strategies.

Finally, we implemented another three PPO variants with different weight-sharing and multi-agent control designs (IPPO, MAPPO, PPO-Joint) and compared performance among them to understand the factors that impacts the performance. Many of the works today focuses on the algorithms itself without paying too much attention on training efficiency and agents’ collaboration behavior, which are the areas our project is focusing on.

Implementation All algorithms are implemented in PyTorch with Unity MLAgents, where each agents observes a 336-dimensional ray-cast state and selects from 6 (striker) or 4 (goalie) discrete actions. All algorithms uses the three-layer MLP backbone $336 \rightarrow 256 \rightarrow 128$ with ReLU activations and role-specific action heads, plus a mirrored value network when required (for PPO variations). Training follows the three-stage curriculum as mentioned above with Adam and Hyperparameter: $\alpha = 3 \times 10^{-4}$, $\gamma = 0.99$, GAE $\lambda = 0.95$, PPO clip $\epsilon = 0.1$, entropy bonus $\beta = 0.001$.

Results Experiment 1 Algorithm Comparison Training a single striker against frozen opponents showed clear separation between methods. PPO quickly improved the average episode reward from the random baseline to near optimal play, while REINFORCE plateaued at a much lower reward and discrete SAC failed to learn a meaningful policy.

Experiment 2 Effectiveness of Staged Training When the same PPO agent progressed through the 3-stage training, the cumulative reward against a random team rose monotonically after each stage.

Experiment 3 Weight-sharing and Control Strategies After staged training, MAPPO (shared critic + independent actors) achieved the highest rating (1506.5); PPO Joint (shared actor + critic) followed, and fully independent PPO trailed. Additional ablation showed that concatenating both agents’ observations into actor policy hurt performance.

Experiment 4 Exploration During Self-play Injecting ϵ -greedy noise in the opponent team produced modest improvements (win-rates 38–48% vs. random), but sample-past opponent replay did not perform well. With a 100-step window it remains below ϵ -greedy variants.

Discussion Decomposing training into progressively harder stages gives dense feedback early, prevents agents from converging on degenerate behaviors and speeds up final convergence. Shared critic stabilizes learning by providing a consistent estimate of joint state value, however, distinct actors allow the striker and goalie to specialize in different action spaces. While we expected opponent replay to diversify experience, naive sample-past hurt performance, likely because stale policies create non-stationary targets.

Conclusion This work demonstrates that a staged training plan combined with carefully chosen weight-sharing algorithm significantly accelerates and stabilizes learning in the multi-agent adversarial game environment.

Staged Training and Various Methods in Multi-agent Game Environment

Chenyang Dai

Department of Computer Science
Stanford University
qddaichy@stanford.edu

Abstract

In this work, we implemented, trained, and evaluated various prominent reinforcement learning algorithms (REINFORCE, Discrete SAC, PPO along with its variations) in a four-agent adversarial game setting called Strikers&Goalies (Juliani et al., 2020). The goal is to understand the effectiveness of staged training approach and experiment with the different agent control strategies for the best performance.

Our work consists of three major parts: 1) Implemented all the algorithms from scratch and investigated the performance differences 2) replaced traditional self-play training approach with staged training approach and demonstrated the effectiveness of the new approach 3) Compared the performance and strategic differences of the algorithms and their variations in the competitive multi-agent environment. Our findings indicate that the staged training strategy accelerates training speed compared to training directly with opponents employing a random strategy or self-play. Additionally, sharing weights encourages the collaborations between agents and enhances performance, while extended periods of self-play generally yield substantial benefits. Our best-performing MAPPO model (a multi-agent PPO model with independent policies and a shared baseline) achieved the top ELO score when competing with other agents.

1 Introduction

Training agents effectively in multi-agent adversarial games can be quite challenging. On the one hand, it requires finding an opponent with suitable skill levels to play against the training agent. If the opponent has exceedingly high or low skill levels, it becomes challenging for the agent to gain rewards and acquire skills, or the agent can overlearn irrelevant behaviors. On the other hand, in a game with mixed interactions and complex group dynamics (both cooperation and competition), it is difficult to set a stable learning objective for each agent as the environment evolves with other agents' behaviors.

This project focuses on developing, training, and evaluating various algorithms for the multi-agent game Strikers&Goalies (Juliani et al., 2020). Our primary goals are improving the traditional self-play idea with a staged training strategy to reduce training time and exploring different model variations and weight-sharing methodologies to encourage agents to learn collaboration and competitive behaviors. We also briefly investigate different strategies to balance exploration and exploitation during training to optimize performance.

This project is driven by our interest in exploring diverse methodologies to simultaneously teach AI agents collaboration and competition, which we believe is a compelling and challenging area in reinforcement learning.

2 Related Work

Self-play as default recipe Since the release of OpenAI’s Competitive Self-Play framework Trapit Bansal (2017), pairing an agent with past or contemporaneous versions of itself has become the default recipe for learning in adversarial games. Because the opponent’s skill level is similar to the learner, self-play automatically generates a progression of increasingly difficult tasks, avoiding the need for manually tuned reward shaping or task generators. Follow-up systems such as AlphaGo Zero demonstrated that, when combined with large-scale compute, self-play can push performance beyond expert human baselines. However, vanilla self-play can suffer from cycling policies or long plateaus if exploration is insufficient.

Deep RL across games A broad spectrum of environments has been used as test-beds for algorithmic ideas. On the low-dimensional end, Mnih et al. (2013) introduced the Deep Q-Network (DQN), which maps raw game frames to joystick actions and exceeds human performance on many Atari 2600 titles. For continuous-control humanoids, Muzio et al. (2022) leveraged a hierarchical RL stack to achieve agile dribbling and racing in RoboCup 3-D, outperforming hand-crafted controllers. In the mixed cooperative–competitive setting, the Multi-Agent Deep Deterministic Policy Gradient (MADDPG) proposed by Lowe et al. (2017) uses centralized critics to counteract non-stationary while allowing decentralized execution, yielding superior coordination in predator–prey and particle-world domains. At the opposite extreme of scale and complexity, the Alpha Star league exploited population-based training, Elo matchmaking and policy distillation to reach Grandmaster level in StarCraft II—surpassing 99.8 % of humans on the official ladder Vinyals et al. (2019).

Parameter sharing in multi-agent PPO variants With the rise of Proximal Policy Optimisation (PPO), multiple authors have investigated how much of the network should be shared among teammates. Independent PPO (IPPO) trains a separate actor–critic per agent, whereas MAPPO couples independent actors with a shared critic that receives joint observations; empirically, the latter delivers stronger coordination on the StarCraft Multi-Agent Challenge and Hanabi Yu et al. (2022). Other work has further explored joint-policy versions, weight factoring, and shared replay buffers.

Our project idea draws idea from previous research efforts, particularly Open AI’s Competitive self-play, the effectiveness of PPO in game setting (MAPPO) Yu et al. (2022) and the DDPG Lowe et al. (2017) framework. In addition, our project draws inspiration from curriculum learning. Automated curriculum learning, as proposed by Matiisen et al. (2019) in Teacher-Student Curriculum Learning (TSCL), involves using another RL agent to propose tasks to the main learning agent. However, many of the works today focuses on the algorithms itself without paying too much attention on training efficiency and agents’ collaboration behavior, which are the areas our project is focusing on. We hope to improve the training speed through a stage training plan while learning more meaningful competition strategy & collaboration behaviors through more effective information sharing among agents in this work.

3 Method

3.1 Problem Formulation

The Strikers&Goalies is a soccer game variant with four agents in two teams. Each team has a Striker and a Goalie. The goal of the Striker agents is to push the ball into the goal area while the Goalie tries to prevent the ball from entering its own goal area. The strikers cannot move into the penalty areas, and the goalies cannot move out of it. In the simulated environment, one game is considered over as soon as any team scores a goal or a pre-defined number of steps has passed and still no goal. Figure 5 depicts the simulation environment.

State Space The environment has continuous state space. Every agent has a 336-dimension state space, corresponding to 11 ray-casts forward distributed over 120 degrees and three ray-casts backward distributed over 90 degrees, each detecting six possible object types (soccer ball, other players, and two goals), along with the object’s distance. This symmetric state space definition enables us to apply learned models to both teams for self-play.

Action Space The environment has discrete action space. For strikers, the action space size is 6, corresponding to forward, backward, left, right, and clockwise and counter-clockwise rotations. For goalies, the action space size is 4, corresponding to forward, backward, left, and right.

Reward Function To account for the difference in responsibilities between the striker and the goalie, the two agents have different reward functions. When the team scores a goal, the striker gains +1 reward and the goalie gains +0.1 reward. When the opponents score a goal, the striker gets a -0.1 reward, and the goalie gets a -1.0 reward. In addition, the goalie has a time penalty of 0.00167 at each time step, while for the striker, we reward it with an existential bonus of 0.00167, as the goalie should keep the game for longer by preventing the opponents from scoring a goal.

Policy We used discrete categorical policies for both Striker and Goalie to take action.

3.2 Algorithms

We implemented the following algorithms from scratch, along with common training tricks such as bonus action entropy, advantages normalization, and generalized advantage estimation (GAE) Schulman et al. (2015) for better advantage evaluation.

Policy Gradient Policy Gradient (REINFORCE) Sutton et al. (1999) is a foundational policy gradient method known for its simplicity and effectiveness. We implemented it as a baseline to assess other algorithms.

Soft Actor-Critic Discrete Soft Actor-Critic (SAC) Christodoulou (2019) is an actor-critic method that alternates between policy evaluation and policy improvement to maximize both the rewards and the policy’s entropy:

$$\pi^* = \pi \sum_{t=0}^T \mathbb{E}_{(s_t, a_t) \sim \tau_\pi} [\gamma^t (r(s_t, a_t) + \alpha H(\pi(\cdot|s_t)))]$$

Under this framework, the value function becomes

$$V(s_t) = \mathbb{E}_{a_t \sim \pi} [Q(s_t, a_t) - \alpha \log(\pi(a_t|s_t))]$$

where α is the temperature parameter.

Proximal Policy Optimization Proximal Policy Optimization (PPO) Schulman et al. (2017) enhances policy gradient approaches by limiting the size of policy updates and promoting stable and efficient learning.

$$L(\theta) = \mathbb{E}_t \left[\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t) \right]$$

where $\hat{A}_t$ is the advantage function, and $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{old}}(a_t|s_t)$.

In the staged training step, we added a bonus entropy term to encourage exploration.

$$L(\theta) = \mathbb{E}_t \left[\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t) + \beta H(\pi_\theta(\cdot|s_t)) \right]$$

Multi-Agent PPO and PPO Joint To experiment with different agent control strategy, we further designed and implemented variations of PPO algorithms. The variations include different ways of sharing weights between agents of the same team, whether to combine states of different agents as inputs to the critic network, and different network architectures and weights sharing as shown in Table 1 and Figure 2. For example, MAPPO uses shared network for critic but two separate networks to control the two players in the game of the same team.

ALGOS	IPPO	MAPPO	PPO (JOINT)
POLICY	INDEPENDENT	INDEPENDENT	SHARED
CRITIC	INDEPENDENT	SHARED	SHARED

Table 1: Policy and Critic sharing with different multi-agent PPO configurations

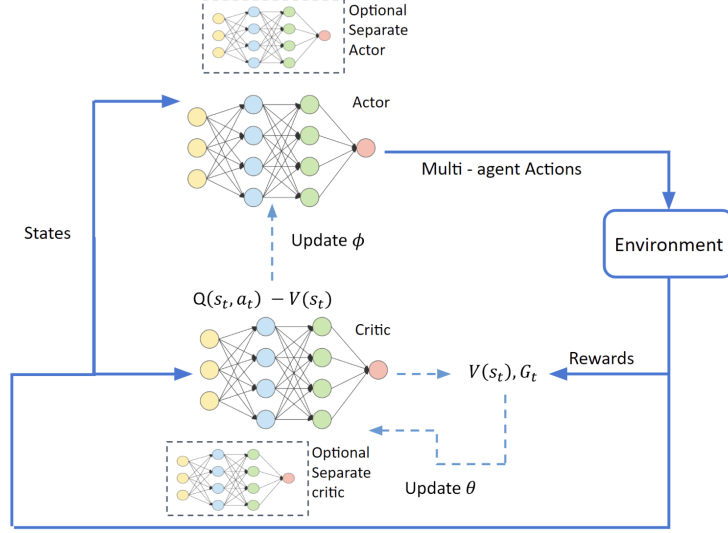


Figure 1: Architecture of PPO and its variations

3.3 Training Strategy

One of the biggest challenges is efficiently training the implemented algorithms with limited computation sources. Our vanilla self-play training showed very slow progress in converging for most algorithms, making us think about better training strategies.

With the heuristics of imitation learning as initialization, we broke down our training process into steps to gradually guide the models through the learning process and hope to speed up the training process.

Stage 1 Training one striker only (other players frozen): We initiated our training process by focusing on a single striker while keeping the other players inactive. This step allows the agent to gain a less noisier reward and solve a simpler task to begin with. We can thus isolate the striker’s learning process from the unpredictable elements introduced by other agents and test the effectiveness of each algorithm. This stage served as a pre-training stage for our agent.

Stage 2 Training two agents in one team (with the other team random) together: Then, we trained both the striker and the goalie of one team simultaneously with random opponents. We utilized combined team rewards to allow specialized learning of strikers and goalies. The presence of a random opponent team continued to provide randomness in gameplay, pushing the agents to react to different game stats and to adapt their strategies.

Stage 3 Training with self-play: In this step, we train our agents with self-play. We intentionally brought various randomness (ϵ -Greedy) to the other team agent’s action to enhance exploration and prevent overfitting to specific strategies. Moreover, we adopted the technique of past opponent sampling with different window sizes, aiming to encourage the team to perform better than in the past.

4 Experimental Setup

4.1 Experiments Goals

Overall, we ran four experiments to evaluate how different agent design and training choices affect final performance in the multi-agent environment.

Experiment 1: Compares the performance among the picked algorithms (Policy gradient, discrete SAC and PPO) with only stage 1 training.

Experiment 2: Evaluates the effectiveness of staged training of the best-performing algorithm from experiment 1.

Experiment 3: Compare performance among different weight-sharing and agent controlling strategies.

Experiment 4: Investigate different strategies to balance exploration and exploitation during training to optimize performance.

4.2 Network Architecture

For a fair comparison, the network used for all algorithms and baseline is 3-layer fully connected layer with ReLU activation. The policy network has an input dimension of the size of the striker or goalie’s observation (336), then with intermediate layers with sizes 256 and 128, followed by the output action dimension of 6 for the striker or 4 for the goalie.

4.3 Hyper Parameters Selection

Learning Rate (α): The learning rate was crucial for training stability. We tested multiple values, ranging from 1e-6 to 1e-2 for various models, and determined that a learning rate of 3e-4 with a decreasing learning rate schedule offered acceptable training performance.

Discount Factor (γ): The discount factor affects how future rewards are valued compared to immediate rewards. We picked 0.99 for all the models because the reward will mostly only be given at the end of an episode.

Entropy Bonus (β): This parameter helps encourage exploration by adding an action entropy bonus term to the reward. After several trials with different values, we set β at 0.001.

Advantage Estimation: We utilized Generalized Advantage Estimation (GAE) with $\lambda = 0.95$ as it provided a good balance between bias and variance in the advantage estimates. We also implemented advantage normalization to stabilize the training process.

Batch Size and Update Frequency: To optimize learning efficiency, we experimented with batch sizes ranging from 32 to 256. A batch size of 32 worked well.

4.4 Performance Evaluation

We used the **winning rate over 500 games** metric for quick comparison against a opponent with fixed policy and the Elo rating system Arpad Elo ([n. d.]) for a more rigorous evaluation. To calculate Elo score, after a match between team A and team B, the score for each team is updated as follows: First, the expected score for each team is calculated based on the current ratings R_A, R_B :

$$E_A = \frac{1}{1 + 10^{(R_B - R_A)/400}}, E_B = \frac{1}{1 + 10^{(R_A - R_B)/400}}$$

Then, the rating for each team is updated

$$R'_A = R_A + K(S_A - E_A), R'_B = R_B + K(S_B - E_B)$$

where K is a constant factor, and S_A, S_B are the scores for each team. $S = 1$ if the team wins, $S = 0.5$ if there is a draw, and $S = 0$ if the team loses.

5 Results

5.1 Experiment 1: Compares Performance Among Picked Algorithms

Figure 2 shows the average episode rewards over training epochs for the picked algorithms in this experiment. The average episode reward for a random player is about -0.2. PPO is able to converge to a strategy that is close to the optimum where it runs straight to the ball and pushes it into the goal while REINFORCE performs not as well. However, PPO still suffers from being stuck at local optima and the agent would not attempt a second kick once its first kick fails. Randomizing the initial position of the ball may be able to mitigate this issue.

Discrete SAC fails to learn a meaningful policy in this stage. We have tried using different entropy scales to control the randomness of the policy. However, the agent will eventually stop moving with a smaller entropy target or act randomly with a larger entropy target. This is likely due to the sparsity of rewards (agents only gain ± 1 reward when scoring a goal). It is difficult to learn value functions using TD residuals compared to using advantage function estimation as in PPO. The algorithm also performs worse than a random agent in the Atari Pong game Christodoulou (2019).

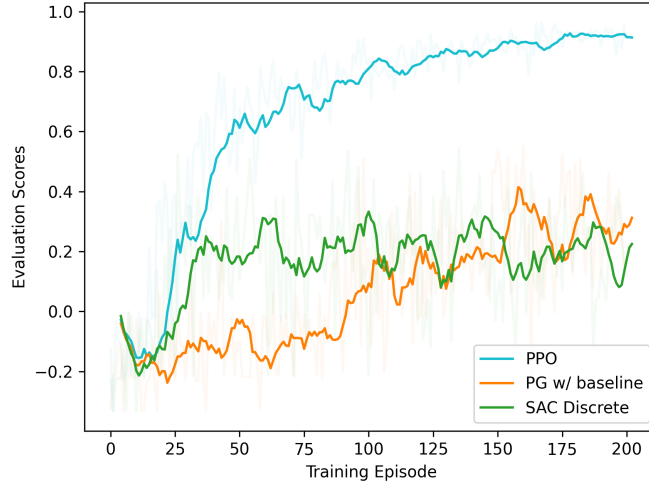


Figure 2: Performance of REINFORCE, Discrete SAC, and PPO over training episodes

5.2 Experiment 2: Effectiveness of Staged Training

After first experiment, we concluded that PPO seems to be more suitable for our problem. In this experiment, we continued training PPO families with step 2 and step 3 as described above. Figure 3 shows the PPO agent’s training progress across stages; the x-axis is the number of training steps across stages, and the y-axis is the team reward against a random opponent team for 100 games.

From this experiment, we concluded that staged training can successfully guide the agent to learn the desired behavior step by step quicker than only self-play. Even though between stages, the agents’ performance will drop at the start, the agent can quickly catch up and continue making progress. We believe this is because the staged training breaks down the difficult game setting into multiple easier tasks, and the agent can learn to achieve each easier task with denser rewards. Also, the behavior learned from the previous stage can help with later stages. As a result, the training can progress faster.

Also, there are two observed caveats about staged training: 1. Agents should not be trained in Stage 1 (training against still opponents) for too long, or it can over-fit by only kicking the ball once and stopping moving. 2. Self-play should be used with a decreasing learning rate and beta (entropy weight) schedule for convergence.

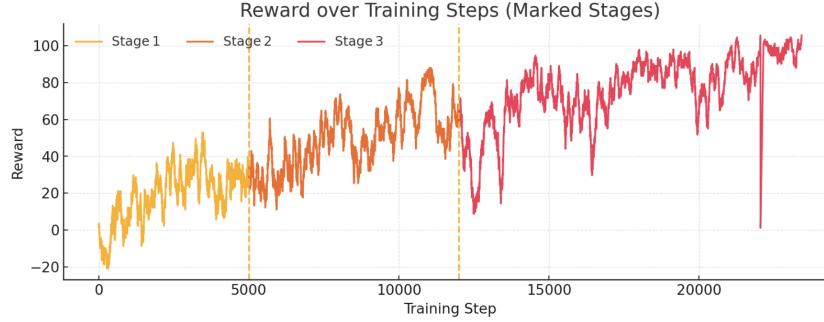


Figure 3: Training progress across stages 1-3, the performance is evaluated by the cumulative reward over 100 matches

5.3 Experiment 3: Weight-sharing and Agent Controlling Strategies

Table 2 shows the Elo scores for different weight-sharing methods after staged training. MAPPO achieves the best performance, PPO (Joint) comes as the second, and IPPO has relative weaker performance compare to the two. We can conclude that a shared critic improves state-value estimation and overall performance, but separate policy networks for the Goalie and Striker still outperform a single shared policy. We hypothesis that this is because the two agents have very different action spaces, giving each its own network lets it specialize in its role, producing better results.

ALGORITHM	MAPPO	PPO (JOINT)	IPPO	SAC	RANDOM
ELO SCORE	1506.52	1489.39	1450.36	1180.63	987.91

Table 2: Elo Ratings of agents for 500 games between each pair

We further investigated the input sharing by training IPPO agents with shared inputs (i.e., each agent has both the striker’s and the goalie’s observation as inputs to the policy network). As shown in Table 3, IPPO with shared input states perform worse than IPPO with local input states, which suggests that introducing other agents’ states makes it difficult for the algorithm to distinguish between the primary and additional observations, leading to sub-optimal performance.

STRIKER	GOALIE	WINS	DRAWS
IPPO (LOCAL STATES)		89%	3%
IPPO (SHARED STATES)		42%	18%
MAPPO		96%	0%
PPO (JOINT)		94%	2%

Table 3: Win rates of agents against random agents

5.4 Experiment 4: Exploration and Exploitation

To further improve the performance gain during self-play training stage, we explored two methods to sample actions for the opponent team, “ ϵ -Greedy” and “Sample Past”. ϵ -Greedy introduces various levels of randomness to the sampled actions. $\epsilon = 1.0$ means all actions would be random, and the opponent team would be the same as Stage 2. $\epsilon = 0.7$ and $\epsilon = 0.3$ mean 70% and 30% of the actions are random respectively, and the others are sampled from policies. In Sample Past, at each rollout for the target team, we sample old parameters with various window lengths for the opponent team. Sample Past would encourage the team to continuously learn by playing with random older versions of itself.

Figure 4 shows the performance of PPO trained with two sampling methods of self-play mentioned above against a team of random agents. We observed both self-play opponent sampling methods

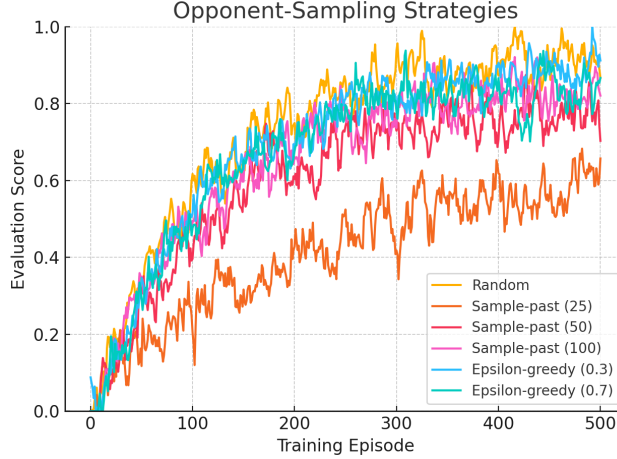


Figure 4: Performance of PPO (Joint) trained with different self-play opponents

didn't provide a significant gain over random action selection. Sample-past with short window length even leads to a significant drop in win rates.

Table 4 reflects the similar results for PPO (Joint) after being trained with self-play for 500 episodes to play against other policies. Training with random opponents still achieves the best-performing policy. The results did not meet our hypothesis that the advanced sampling method could further improve the policy. In the future, we will continue to investigate proper sampling strategies in this environment.

OPPONENT AGENT	TEAM 1 (BLUE)		
	PPO	PPO(JOINT)	MAPPO
ϵ -GREEDY(1.0)	42%	48%	40%
ϵ -GREEDY(0.3)	38%	41%	37%
ϵ -GREEDY(0.7)	40%	44%	43.5%
SAMPLE-PAST(25)	6.5%	6%	6%
SAMPLE-PAST(50)	27%	31%	24.5%
SAMPLE-PAST(100)	25%	38.5%	28%

Table 4: Win rates of PPO (Joint) (trained for 500 episodes) with different self-play strategies

6 Discussion and Conclusion

In this project, we explored, implemented, and trained several RL algorithms and their variations from scratch to play the Striker&Goalie game. We experimented with different training strategies, sampling strategies, and model variations. Our experiments derived the following insights:

- PPO and its variations outperform other algorithms in terms of both converge speed and performance. Our MAPPO agent achieved the best ELO rating and learned noticeable collaboration behavior when playing the game.
- Training strategy is critical for game settings: using a staged training strategy can solve the slow start issue when training the vanilla models by dividing the complicated problems into sub-tasks for the agent to learn.
- Shared baseline/critic can encourage collaboration between agents and improve team performance. However, separate the decision making networks for two agents have a little gain in our setting.
- Sampling strategy is critical for self-play: introducing more variance in opponents can lead to faster convergence and a more robust policy.

In conclusion, our work demonstrates that a staged training plan combined with carefully chosen weight sharing algorithm greatly accelerates and stabilizes learning in the multi-agent adversarial game environment.

There are still areas for improvement in our project. For Discrete SAC, we could try N-step estimators for the target value function and use a prioritized replay buffer to put more weight on experiences where the striker scores a goal. Moreover, when running matches between different agents, we observed that the strikers learn to score goals, but the goalies barely learn to defend the gates. A better (extrinsic) reward function or incorporating intrinsic reward Burda et al. (2018) could encourage the agents to explore different actions and learn a better policy.

7 Contribution

N/A, one person group.

Changes from Proposal There’s minor changes from project proposal. After implementing first two algorithms, I realized that training every algorithm is time consuming. To focus more on training and experimenting, I decided to stop investing in two algorithms, namely DDPG, MADDPG which were originally planned.

References

- Wikipedia Arpad Elo. [n.d.]. Elo rating system. https://en.wikipedia.org/wiki/Elo_rating_system.
- Yuri Burda, Harri Edwards, Deepak Pathak, Amos Storkey, Trevor Darrell, and Alexei A Efros. 2018. Large-scale study of curiosity-driven learning. *arXiv preprint arXiv:1808.04355* (2018).
- Petros Christodoulou. 2019. Soft actor-critic for discrete action settings. *arXiv preprint arXiv:1910.07207* (2019).
- Arthur Juliani, Vincent-Pierre Berges, Ervin Teng, Andrew Cohen, Jonathan Harper, Chris Elion, Chris Goy, Yuan Gao, Hunter Henry, Marwan Mattar, and Danny Lange. 2020. Unity: A general platform for intelligent agents. *arXiv preprint arXiv:1809.02627* (2020).
- Ryan Lowe, Yi I Wu, Aviv Tamar, Jean Harb, OpenAI Pieter Abbeel, and Igor Mordatch. 2017. Multi-agent actor-critic for mixed cooperative-competitive environments. *Advances in neural information processing systems* 30 (2017).
- Tambet Matiisen, Avital Oliver, Taco Cohen, and John Schulman. 2019. Teacher–student curriculum learning. *IEEE transactions on neural networks and learning systems* 31, 9 (2019), 3732–3740.
- Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. 2013. Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602* (2013).
- Alexandre FV Muzio, Marcos ROA Maximo, and Takashi Yoneyama. 2022. Deep reinforcement learning for humanoid robot behaviors. *Journal of Intelligent & Robotic Systems* 105, 1 (2022), 12.
- John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. 2015. High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438* (2015).
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal Policy Optimization Algorithms. *arXiv preprint arXiv:1707.06347* (2017).
- Richard S Sutton, David McAllester, Satinder Singh, and Yishay Mansour. 1999. Policy gradient methods for reinforcement learning with function approximation. *Advances in neural information processing systems* 12 (1999).

Szymon Sidor Ilya Sutskever Igor Mordatch Trapit Bansal, Jakub Pachocki. 2017. Emergent Complexity via Multi-Agent Competition. *arXiv preprint arXiv:1710.03748* (2017).

Oriol Vinyals, Igor Babuschkin, Wojciech M Czarnecki, Michaël Mathieu, Andrew Dudzik, Junyoung Chung, David H Choi, Richard Powell, Timo Ewalds, Petko Georgiev, et al. 2019. Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature* 575, 7782 (2019), 350–354.

Chao Yu, Akash Velu, Eugene Vinitsky, Jiaxuan Gao, Yu Wang, Alexandre Bayen, and Yi Wu. 2022. The Surprising Effectiveness of PPO in Cooperative, Multi-Agent Games. *arXiv:2103.01955 [cs.LG]*

A Appendix

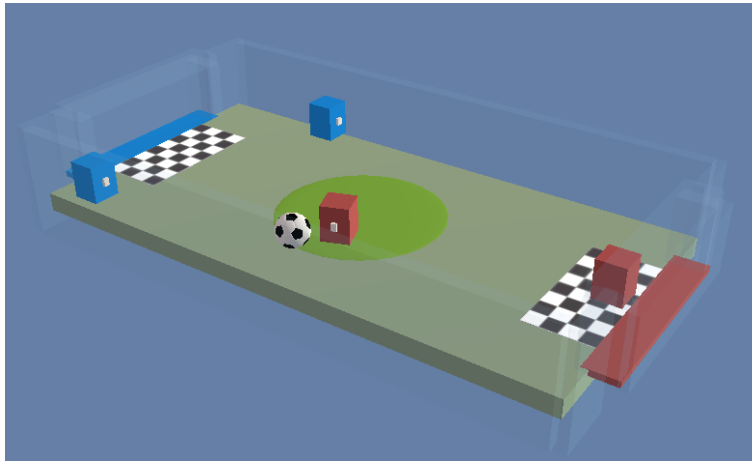


Figure 5: Simulator environment